MSc Data Science

DS601: Mathematical Foundations for Data Science

Tutorial 4 - Complete Solutions

Date: 22/08/2026

This solution follows the two-part structure of the tutorial: analytical calculations in Section A and Python-based computational verification and scaling experiments in Section B.

Section A: Analytical Questions

Q1) Binomial Distribution

Given

$$X \sim \text{Binomial}(n = 4, p = 0.85), \qquad q = 1 - p = 0.15.$$

(a) Binomial PMF

For exactly x successes in n independent Bernoulli experiments,

$$\boxed{\mathbb{P}(X = x) = \binom{n}{x} p^x (1-p)^{n-x}}, \qquad x = 0, 1, \ldots, n.$$

Here, $n = 4$ and $p = 0.85$, so

$$\mathbb{P}(X = x) = \binom{4}{x} (0.85)^x (0.15)^{4-x}.$$

(b) Possible values of X

Since there are four experiments, the number of successes can be

$$\boxed{X \in \{0, 1, 2, 3, 4\}}.$$

(c) Probabilities for all possible outcomes

$$\mathbb{P}(X = 0) = \binom{4}{0} (0.85)^0 (0.15)^4 = 0.00050625,$$
$$\mathbb{P}(X = 1) = \binom{4}{1} (0.85)^1 (0.15)^3 = 0.01147500,$$
$$\mathbb{P}(X = 2) = \binom{4}{2} (0.85)^2 (0.15)^2 = 0.09753750,$$
$$\mathbb{P}(X = 3) = \binom{4}{3} (0.85)^3 (0.15)^1 = 0.36847500,$$
$$\mathbb{P}(X = 4) = \binom{4}{4} (0.85)^4 (0.15)^0 = 0.52200625.$$

Binomial probabilities for $n = 4$ and $p = 0.85$

Number of successes, x	$\mathbb{P}(X = x)$
0	0.00050625
1	0.01147500
2	0.09753750
3	0.36847500
4	0.52200625

(d) Sum of all probabilities

$$\begin{aligned}&\mathbb{P}(X=0)+\mathbb{P}(X=1)+\mathbb{P}(X=2)+\mathbb{P}(X=3)+\mathbb{P}(X=4)\\&=0.00050625+0.011475+0.0975375+0.368475+0.52200625\\&=\boxed{1.00000000}.\end{aligned}$$

The outcomes $X = 0, 1, 2, 3, 4$ are mutually exclusive and collectively exhaustive: exactly one of them must occur. Therefore the total probability over the complete sample space must equal 1.

(e) Maximum probability

From the table,

$$\boxed{\max_x \mathbb{P}(X = x) = 0.52200625}.$$

(f) Location of the maximum

The maximum occurs at

$$\boxed{x = 4}.$$

Hence, the most likely outcome is **4 successes in 4 experiments**.

(g) Why larger numbers of successes are more likely

A single trial succeeds with probability 0.85, which is much larger than the failure probability 0.15. Consequently, sequences containing many successes carry more probability mass than sequences containing many failures. This is reflected in the distribution: the probabilities increase strongly toward $x = 3$ and $x = 4$, with $x = 4$ being the most likely outcome.

Q2) Continuous Probability Distribution

The probability density function is

$$f(x) = 6x(1 - x), \qquad 0 \le x \le 1,$$

and $f(x) = 0$ outside $[0, 1]$. The specified spacing is $\Delta x = 0.1$.

(a) Values of $f(x)$

For each grid point,

$$f(x) = 6x(1 - x).$$

Density values for $\Delta x = 0.1$

x	$f(x) = 6x(1-x)$
0.0	0.00
0.1	0.54
0.2	0.96
0.3	1.26
0.4	1.44
0.5	1.50
0.6	1.44
0.7	1.26
0.8	0.96
0.9	0.54
1.0	0.00

(b) Maximum density and its location

The largest tabulated value is

$$\boxed{f_{\max} = 1.5 \quad \text{at} \quad x = 0.5}.$$

This also agrees with calculus: $f'(x) = 6 - 12x = 0$ gives $x = 0.5$, and $f''(x) = -12 < 0$ confirms a maximum.

(c) Approximate total probability

Using the requested sum,

$$S = \sum_i f(x_i)\Delta x.$$

The sum of the density values is

$$\sum_i f(x_i) = 9.9.$$

Therefore

$$S = (9.9)(0.1) = \boxed{0.99}.$$

The result is not exactly 1 because the integral is being approximated using a finite grid. The exact total probability is

$$\int_0^1 6x(1-x)\,dx = \left[3x^2 - 2x^3\right]_0^1 = 1.$$

(d) Comparing $\sum_i f(x_i)$ and $\sum_i f(x_i)\Delta x$

Here,

$$\sum_i f(x_i) = \boxed{9.9}, \qquad \sum_i f(x_i)\Delta x = \boxed{0.99}.$$

The second quantity approximates total probability because probability over a small interval of width Δx is approximately $f(x_i)\Delta x$. Density values themselves are not probabilities; they have to be integrated over an interval. A probability density can even be larger than 1 at individual points, as happens here at $x = 0.5$ where $f(0.5) = 1.5$.

(e) Probability at one exact point

For a continuous random variable,

$$\boxed{\mathbb{P}(X = 0.5) = 0}.$$

The density value $f(0.5) = 1.5$ measures local density, not point probability. Continuous probabilities are obtained from areas under the density curve:

$$\mathbb{P}(a \leq X \leq b) = \int_a^b f(x)\,dx.$$

A single point has zero width and therefore zero probability.

Section B: Computational Questions

Q3) Computational Analysis of the Binomial Distribution

Part I: Reproduce the analytical result for $n = 4$

The following program directly implements the binomial PMF.

```
import math

p = 0.85
n = 4

probabilities = []
for x in range(n + 1):
    prob = math.comb(n, x) * (p ** x) * ((1 - p) ** (n - x))
    probabilities.append(prob)
    print(f"x = {x}, P(X=x) = {prob:.8f}")

x_peak = max(range(n + 1), key=lambda x: probabilities[x])
max_prob = probabilities[x_peak]
S = sum(probabilities)

print("x_peak =", x_peak)
print("Maximum probability =", max_prob)
print("Sum of probabilities =", S)
```

The computed values are the same as the analytical values in Q1, up to floating-point representation. The maximum occurs at $x = 4$, with probability 0.52200625. The total may print as a value such as 0.9999999999999999 rather than exactly 1.0 because most decimal numbers cannot be represented exactly in binary floating-point arithmetic.

Part II: Increase the number of experiments

The following code performs the requested calculations for $n = 4, 10, 20, 50, 100$.

```
import math

p = 0.85
n_values = [4, 10, 20, 50, 100]

for n in n_values:
    probs = [
        math.comb(n, x) * (p ** x) * ((1 - p) ** (n - x))
        for x in range(n + 1)
    ]

    x_peak = max(range(n + 1), key=lambda x: probs[x])
    max_prob = probs[x_peak]
    ratio = x_peak / n
    total = sum(probs)

```

```
    print(n, x_peak, max_prob, ratio, total)
```

Computational binomial results for $p = 0.85$

n	x_{peak}	**Maximum probability**	x_{peak}/n	**Sum of probabilities**
4	4	0.52200625	1.000	1.000000000000
10	9	0.34742542	0.900	1.000000000000
20	17	0.24282890	0.850	1.000000000000
50	43	0.15745276	0.860	1.000000000000
100	85	0.11109096	0.850	1.000000000000

(h) Plotting the distributions

```
import math
import numpy as np
import matplotlib.pyplot as plt

p = 0.85

for n in [4, 20, 50, 100]:
    x = np.arange(n + 1)
    probs = [
        math.comb(n, int(k)) * p**int(k) * (1-p)**(n-int(k))
        for k in x
    ]
    plt.plot(x, probs, marker='o', markersize=3, label=f"n = {n}")

plt.xlabel("Number of successes, x")
plt.ylabel("P(X = x)")
plt.title("Binomial distributions for p = 0.85")
plt.legend()
plt.grid(alpha=0.25)
plt.show()
```

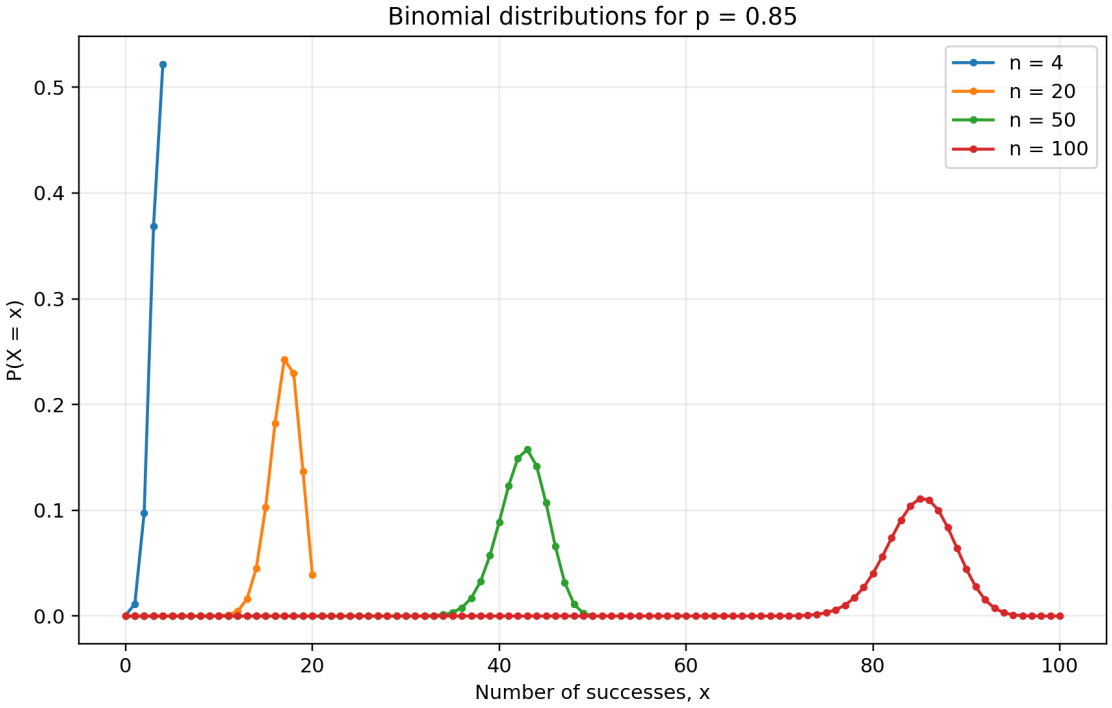


Figure 1: Binomial PMFs for $p = 0.85$ as the number of experiments increases.

(i) Observation about the peak

The most likely count is close to np. More precisely, a binomial mode is generally near $(n+1)p$, subject to the fact that x must be an integer. Consequently, the ratio x_{peak}/n approaches the success probability $p = 0.85$ as n becomes large. The small deviations, such as 0.86 for $n = 50$, are caused by integer rounding of the peak location.

Statistically, this reflects the concentration of the observed success fraction X/n around the true success probability p as the number of independent trials increases. This is consistent with the law of large numbers.

(j) Why the computed sum may not be exactly 1

Computers usually store real numbers using finite-precision binary floating-point representation. Values such as 0.85, powers of 0.85, and repeated additions cannot generally be represented exactly. Tiny rounding errors accumulate during the calculation, so a theoretically exact total of 1 may appear as, for example, 0.9999999999999999 or 1.0000000000000002. Such differences are numerical round-off, not a failure of the probability model.

Q4) Computational Analysis of the Continuous Probability Distribution

Part I: Reproduce the analytical result

```
import numpy as np

Dx = 0.1
x = np.arange(0, 1 + Dx/2, Dx)
f = 6 * x * (1 - x)

for xi, fi in zip(x, f):
    print(f"x = {xi:.1f}, f(x) = {fi:.2f}")

index_peak = np.argmax(f)
```

```
x_peak = x[index_peak]
f_max = f[index_peak]
S = np.sum(f) * Dx

print("x_peak =", x_peak)
print("Maximum f(x) =", f_max)
print("Approximate total probability =", S)
```

The computed grid values reproduce Q2. The numerical maximum is $f_{\max} = 1.5$ at $x = 0.5$, and the requested sum gives $S = 0.99$ for $\Delta x = 0.1$.

Part II: Increase the numerical resolution

```
import numpy as np

for Dx in [0.1, 0.05, 0.01, 0.001]:
    x = np.arange(0, 1 + Dx/2, Dx)
    f = 6 * x * (1 - x)

    index_peak = np.argmax(f)
    x_peak = x[index_peak]
    f_max = f[index_peak]

    S1 = np.sum(f)
    S2 = np.sum(f) * Dx

    print(
        "Dx =", Dx,
        "points =", len(x),
        "x_peak =", x_peak,
        "f_max =", f_max,
        "S1 =", S1,
        "S2 =", S2
    )
```

Effect of grid spacing on the numerical approximation

Δx	Number of points	x_{peak}	Maximum $f(x)$	Approx. total probability
0.1	11	0.5	1.5	0.990000
0.05	21	0.5	1.5	0.997500
0.01	101	0.5	1.5	0.999900
0.001	1001	0.5	1.5	0.999999

(g) Plotting different resolutions

```
import numpy as np
import matplotlib.pyplot as plt

for Dx in [0.1, 0.05, 0.01, 0.001]:
    x = np.arange(0, 1 + Dx/2, Dx)
    f = 6 * x * (1 - x)
    plt.plot(x, f, label=f"Delta x = {Dx}")

plt.xlabel("x")
plt.ylabel("f(x) = 6x(1-x)")
plt.title("Effect of numerical resolution")
plt.legend()
```

```
plt.grid(alpha=0.25)
plt.show()
```

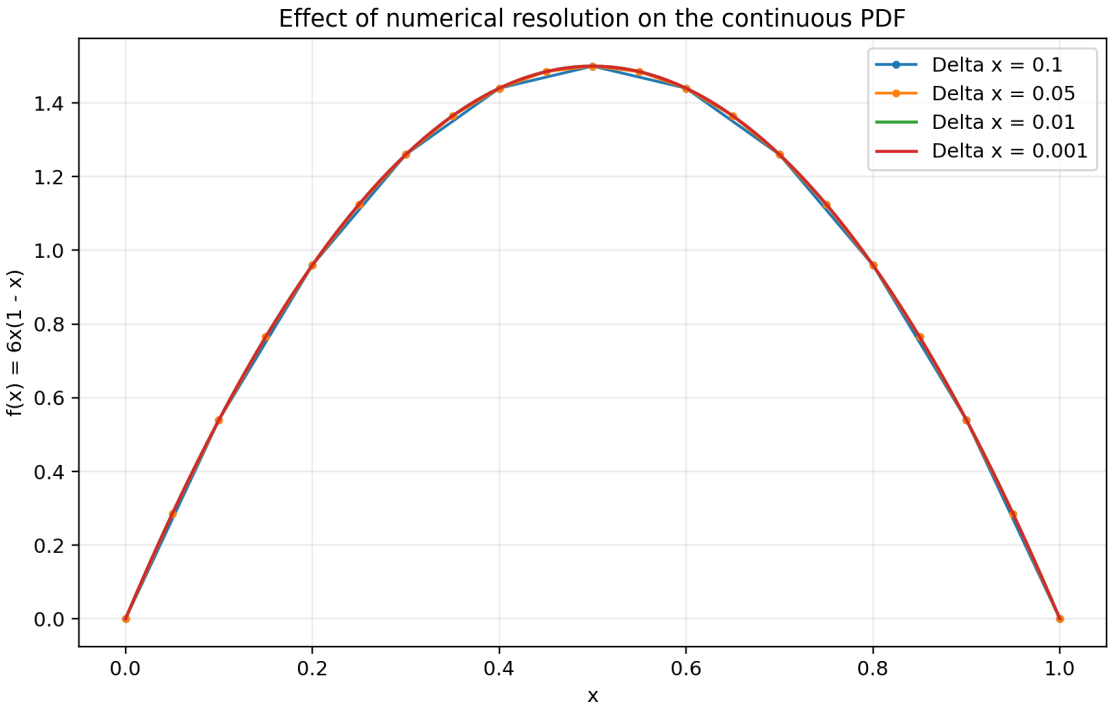


Figure 2: As Δx decreases, the sampled curve becomes a finer representation of the same continuous density.

(h) Comparing S_1 and S_2

For the requested grid spacings:

Δx	$S_1 = \sum_i f(x_i)$	$S_2 = \sum_i f(x_i)\Delta x$
0.1	9.900	0.990000
0.05	19.950	0.997500
0.01	99.990	0.999900
0.001	999.999	0.999999

It is $\boxed{S_2}$ that approaches 1. As the spacing becomes smaller, more density samples are included, so the unscaled sum S_1 grows approximately like $1/\Delta x$. Multiplication by Δx converts each sampled density value into an approximate probability contribution over a small interval.

(i) Resolution, accuracy, number of points, and computational cost

Reducing Δx improves numerical resolution because the continuous curve is sampled at more closely spaced points. For a fixed interval $[0, 1]$, the number of points is approximately

$$N \approx \frac{1}{\Delta x} + 1.$$

Thus, decreasing Δx generally improves the approximation but increases memory usage and the number of arithmetic operations. For example, reducing Δx from 0.1 to 0.001 increases the number of points from 11 to 1001. Therefore numerical work involves a trade-off: finer grids usually improve accuracy but cost more computation.

(j) Why numerical integration is approximated by a weighted sum

An integral represents the accumulated area under a function. Dividing $[a, b]$ into many small intervals of width Δx, the area of one narrow rectangle is approximately

$$f(x_i)\Delta x.$$

Adding all such contributions gives a Riemann-sum approximation:

$$\boxed{\int_a^b f(x)\,dx \approx \sum_i f(x_i)\Delta x}.$$

As $\Delta x \to 0$, the rectangles become narrower and the Riemann sum converges to the exact integral under suitable regularity conditions. Multiplication by Δx is therefore essential because $f(x_i)$ is a density (height), while $f(x_i)\Delta x$ represents an area and hence an approximate probability.

Final Observation

For a **discrete** random variable, probability is assigned directly to individual outcomes, so total probability is obtained by summing the probability mass function:

$$\boxed{\sum_x \mathbb{P}(X = x) = 1}.$$

For a **continuous** random variable, individual points have probability zero. Probability is obtained from area under the density curve, so total probability is calculated by integration:

$$\boxed{\int_{-\infty}^{\infty} f(x)\,dx = 1}.$$

Numerically, the integral is approximated using weighted sums such as

$$\sum_i f(x_i)\Delta x,$$

not by simply adding the density values. This distinction between probability mass and probability density is the central difference illustrated by Q1-Q4.